

# Datové struktury a algoritmy

**Přednáška 5**, poznámky

**Jan Voráček**

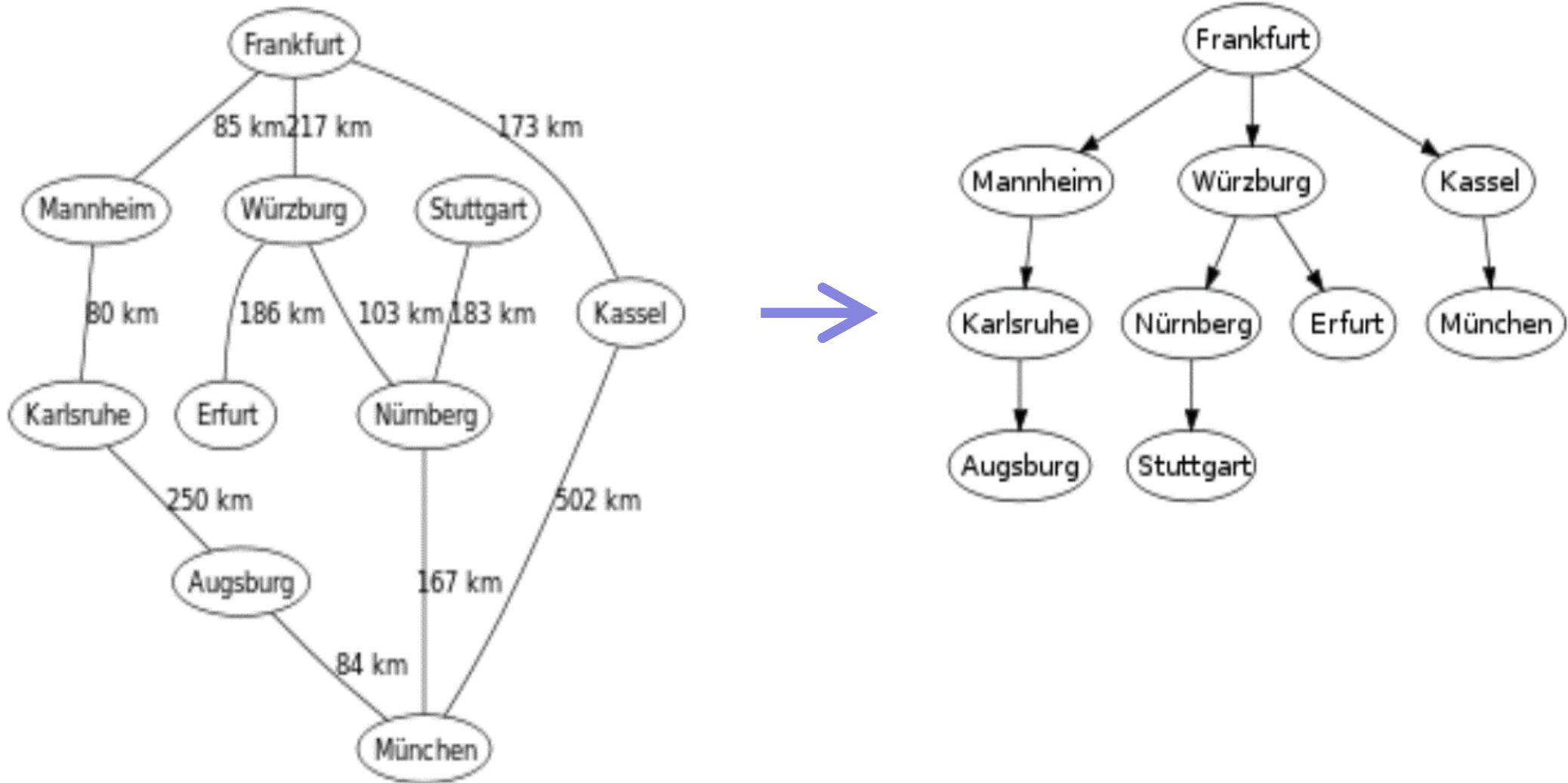
VŠPJ Jihlava, voracek@vspj.cz

# Administrativa a dnešní témata

---

- Prezentace (rezervováno/odevzdáno/přítomno): 3/1/?
  - Master (Akra-Bazzi) teorém
  - Binární stromy
  - Prioritní fronta
- Bonifikovaný kvíz 1: Výpočetní složitost algoritmů
- Stromy a jejich obecné vlastnosti (dokončení)
- Metody prohledávání stromů a grafů
- Vzorové opakovací příklady zkouškového typu
- Halda a prioritní fronta (úvod)
- *Bonifikovaný kvíz 2 (bude-li čas): Stromy a jejich obecné vlastnosti*

# Převod grafu na strom pomocí metody DFS



Zdroj: [https://en.wikipedia.org/wiki/Breadth-first\\_search](https://en.wikipedia.org/wiki/Breadth-first_search)

# Srovnání průchodů stromem a grafem

Do hloubky

## Průchody stromem

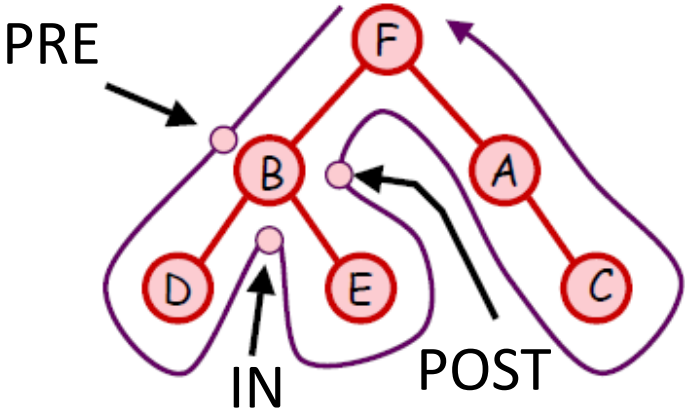
PREorder (FBDEAC)

INorder (DBEFAC)

POSTorder (DEBCAF)

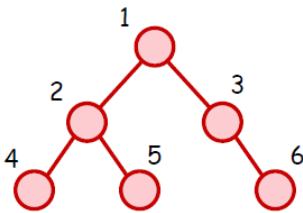
## Průchody grafem

DFS (stack)



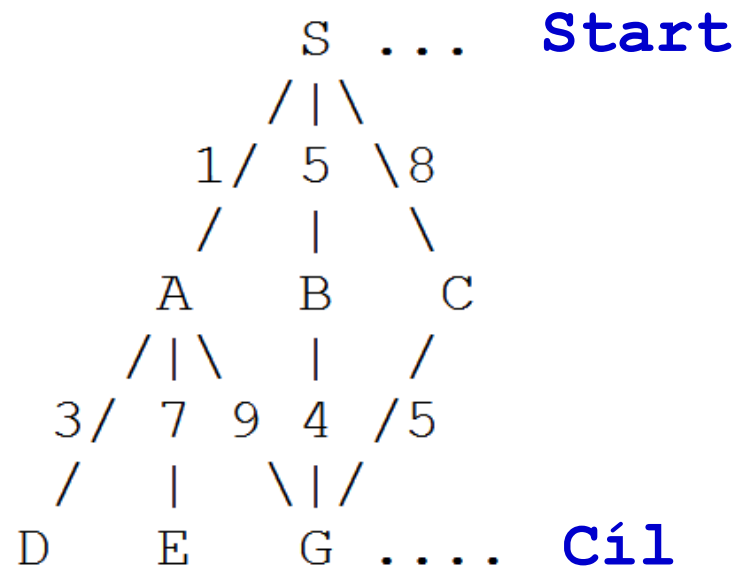
Do šířky

Level order



BFS (queue)

# Prohledávání grafu

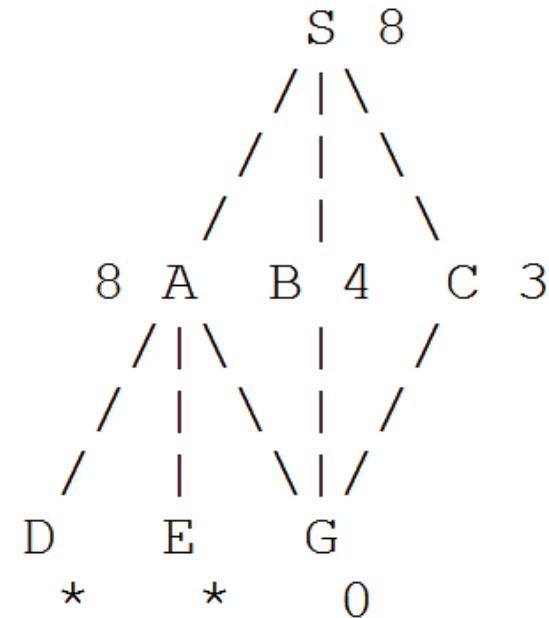


$g(n)$ : ohodnocení hran

**Neinformované:** pouze  $g(n)$

- BFS
- DFS

+



$h(n)$ : heuristika

**Informované:**  $h(n)$ ,  $g(n) + h(n)$

- Greedy
- A\*

# Neinformované prohledávání grafu, DFS

- Implementace: LIFO, tj. **zásobník**
- Složitost:  $O(b^d)$ , kde  $b$  je faktor větvení a  $d$  je hloubka

Expandovaný uzel

Seznam uzlů ke zpracování

{ S }

S

{ A B C }

A

{ D E G B C }

D

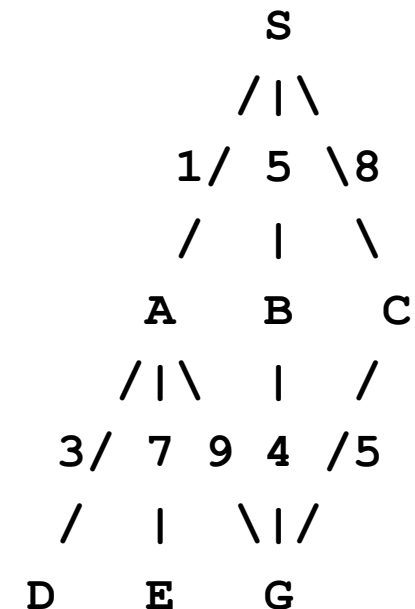
{ E G B C }

E

{ G B C }

G

{ B C }



# Neinformované prohledávání grafu, BFS

- Implementace: FIFO, tj. **fronta**
- Složitost:  $O(b^d)$  kde  $b$  je faktor větvení a  $d$  je hloubka

Expandovaný uzel

-----

S

A

B

C

D

E

G

Seznam uzlů ke zpracování

-----

{ S }

{ A B C }

{ B C D E G }

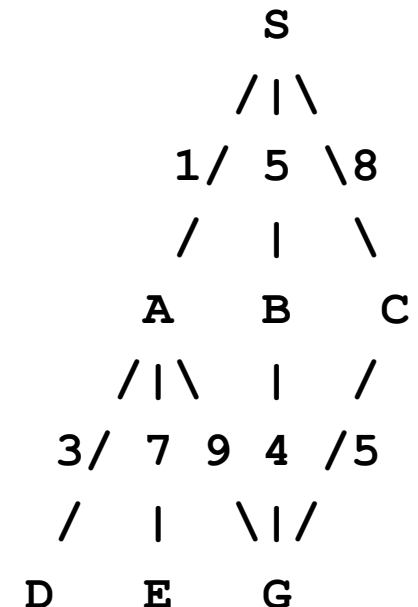
{ C D E G G' }

{ D E G G' G'' }

{ E G G' G'' }

{ G G' G'' }

{ G' G'' }



# Informované prohledávání grafu, greedy

- Postup ve směru největší okamžité výhody bez možnosti návratu (antigradientní)
  - $f(n) = h(n)$
  - Třídí uzly v seznamu podle hodnoty  $h(n)$
  - Alternativní název: Hill climbing
- Implementace: **P-FIFO** (prioritní fronta)

Expandovaný uzel

Seznam uzlů

S

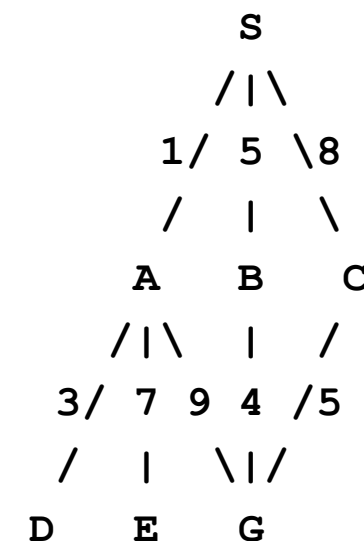
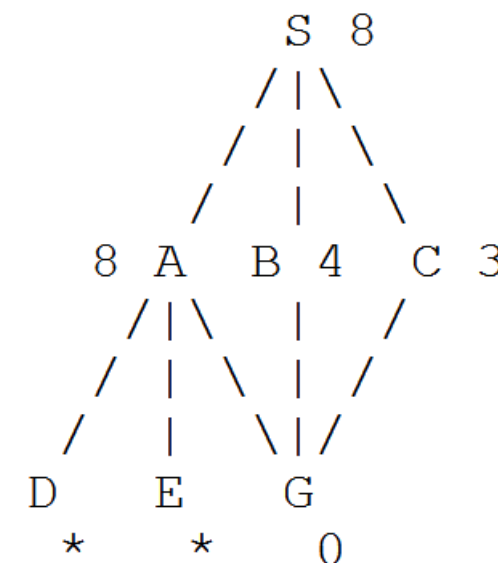
{ S(8) }

C

{ G(0) B(4) A(8) }

G

{ B(4) A(8) }



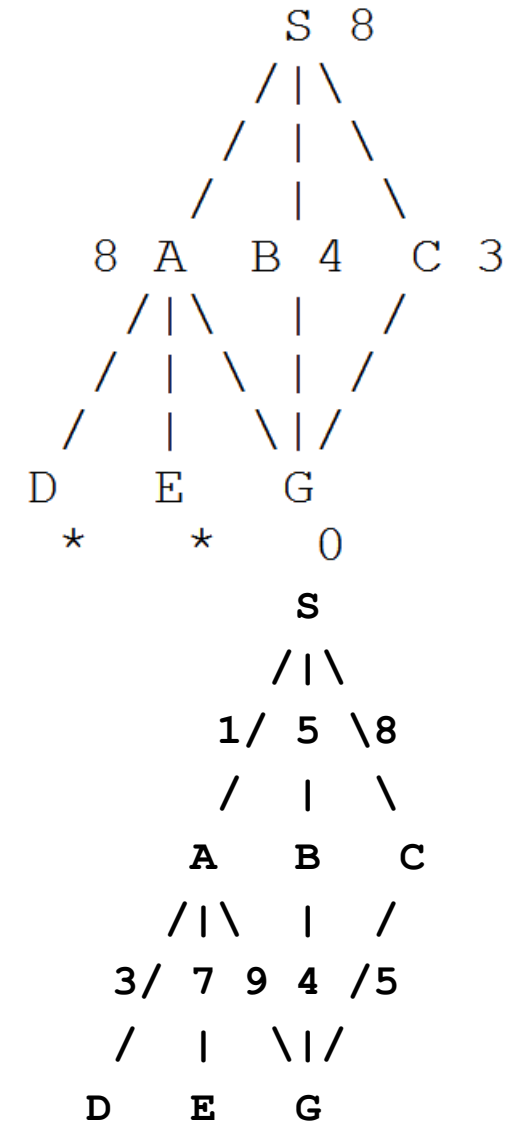


# Informované prohledávání grafu, A\*

- Zohledňuje užitkovou funkci i heuristiku
  - $f(n) = g(n) + h(n)$
  - Je rychlý a optimální ve smyslu heuristiky

Implementace: **P-FIFO** (prioritní fronta)

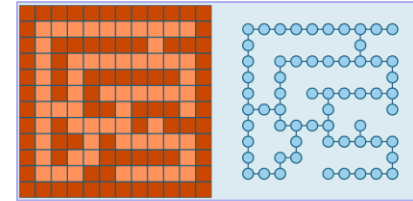
| Exp. uzel | Seznam uzlů                        |
|-----------|------------------------------------|
| -----     | -----                              |
|           | { S(8) }                           |
| S         | { A(9) B(9) C(11) }                |
| A         | { B(9) C(11) G(17) D(inf) E(inf) } |
| B         | { G(8) C(11) G(17) D(inf) E(inf) } |
| G         | { C(11) G(17) D(inf) E(inf) }      |



# Ilustrativní příklad: Bludiště

---

- Jde o úlohu hledání (jakékoli) cesty v grafu, probíhající obvykle v následujících krocích:



1. Vytvoření grafu, reprezentujícího řešený problém
  - ADT mapa (klíče: souřadnice, hodnoty sousedé)
2. Prohledávání grafu
  - Do hloubky – aktuální postup vždy pouze jednou cestu (zásobník nebo rekurze)
  - Do šířky – aktuální postup všemi směry, což může být časově náročnější (fronta)
  - A\* - optimalizované prohledávání, prioritní fronta

# Bludiště, DFS

---

- Vytvoř seznam navštívených vrcholů (souřadnic) [SNV]
- Vytvoř zásobník vrcholů (souřadnic), které lze navštívit
- Dokud není zásobník prázdný:
  - Vyjmi a vyšetři prvek na vrcholu zásobníku
  - Je-li to cílový vrchol, skonči
  - Je-li v SNV, ignoruj ho
  - Ulož prvek do SNV a všechny jeho sousedy ulož do zásobníku
- Dojde-li k vyprázdnění zásobníku bez nalezení cíle, úloha nemá řešení

# Bludiště, BFS

---

- Vytvoř seznam navštívených vrcholů (souřadnic) [SNV]
- Vytvoř frontu vrcholů (souřadnic), které lze navštívit
- Dokud není fronta prázdná:
  - Vyjmi a vyšetři výstupní prvek fronty
  - Je-li to cílový vrchol, skonči
  - Je-li v SNV, ignoruj ho
  - Ulož prvek do SNV a všechny jeho sousedy ulož do fronty
- Dojde-li k vyprázdnění fronty bez nalezení cíle, úloha nemá řešení

# Zásobník, příklady

---

- Seznam stránek v prohlížeči (vpřed, vzad)
- Editace obsahu příkazového řádku
- Aktivační záznamy funkcí při překladu a běhu programu Párování programovacích struktur, typicky závorek
- Převod z infixového zápisu do postfixového
- Vyhodnocování postfixového zápisu (stack machine)

# Fronta/prioritní fronta, příklady

---

- Fronty jakéhokoli typu (tisk, procesor, síť, algoritmizace teoretických i praktických problémů)
- Používáme je v případech, kdy je třeba zohlednit časové hledisko (sekvenci příchodů)

## $O(f(n))$ u pole, spojového seznamu a stromů

---

|              | Hledání | Vkládání | Rušení |
|--------------|---------|----------|--------|
| Pole         |         |          |        |
| SS (začátek) |         |          |        |
| SS (konec)   |         |          |        |
| BS           |         |          |        |
| BVS          |         |          |        |
| VBVS         |         |          |        |

## $O(f(n))$ u pole, spojového seznamu a stromů, řešení

---

|              | Hledání      | Vkládání     | Rušení       |
|--------------|--------------|--------------|--------------|
| Pole         | $O(n)$       | $O(n)$       | $O(n)$       |
| SS (začátek) | $O(n)$       | $O(1)$       | $O(1)$       |
| SS (konec)   | $O(n)$       | $O(n)$       | $O(n)$       |
| BS           | $O(n)$       | $O(n)$       | $O(n)$       |
| BVS          | $O(h)$       | $O(h)$       | $O(h)$       |
| VBVS         | $O(\log(n))$ | $O(\log(n))$ | $O(\log(n))$ |



# Rekurze a výpočetní složitost, příklad

---

Co bude **výstupem** následujícího programu pro  $i = 4$ ?

```
void f(int i) {  
    if (i > 1) {  
        f(i/2);  
        f(i/2);  
    }  
    cout << "#";  
}
```

Jaká je jeho  **$O(n)$** ?

# Příklady, rekurze → Master theorem

BINÁRNÍ VYHLEDÁVÁNÍ

$$T(n) = T\left(\frac{n}{2}\right) + O(1) \quad a=1 \quad b=2 \quad d=0 \quad \boxed{O(\log n)}$$

SEKVENČNÍ VYHLEDÁVÁNÍ

$$T(n) = T(n-1) + O(1) \quad a=1 \quad b=1 \quad d=0 \quad \boxed{O(n)}$$

PRIMOD STRUČNĚ

$$T(n) = 2T(n/2) + O(1) \quad a=2 \quad b=2 \quad d=0 \quad \boxed{O(n)}$$

SELECT SORT

$$T(n) = T(n-1) + O(n) \quad a=1 \quad b=1 \quad d=1 \quad \boxed{O(n^2)}$$

MERGE SORT

$$T(n) = T\left(\frac{n}{2}\right) + O(n) \quad \boxed{O(n \log n)}$$



# BS, výpočetní složitost

---

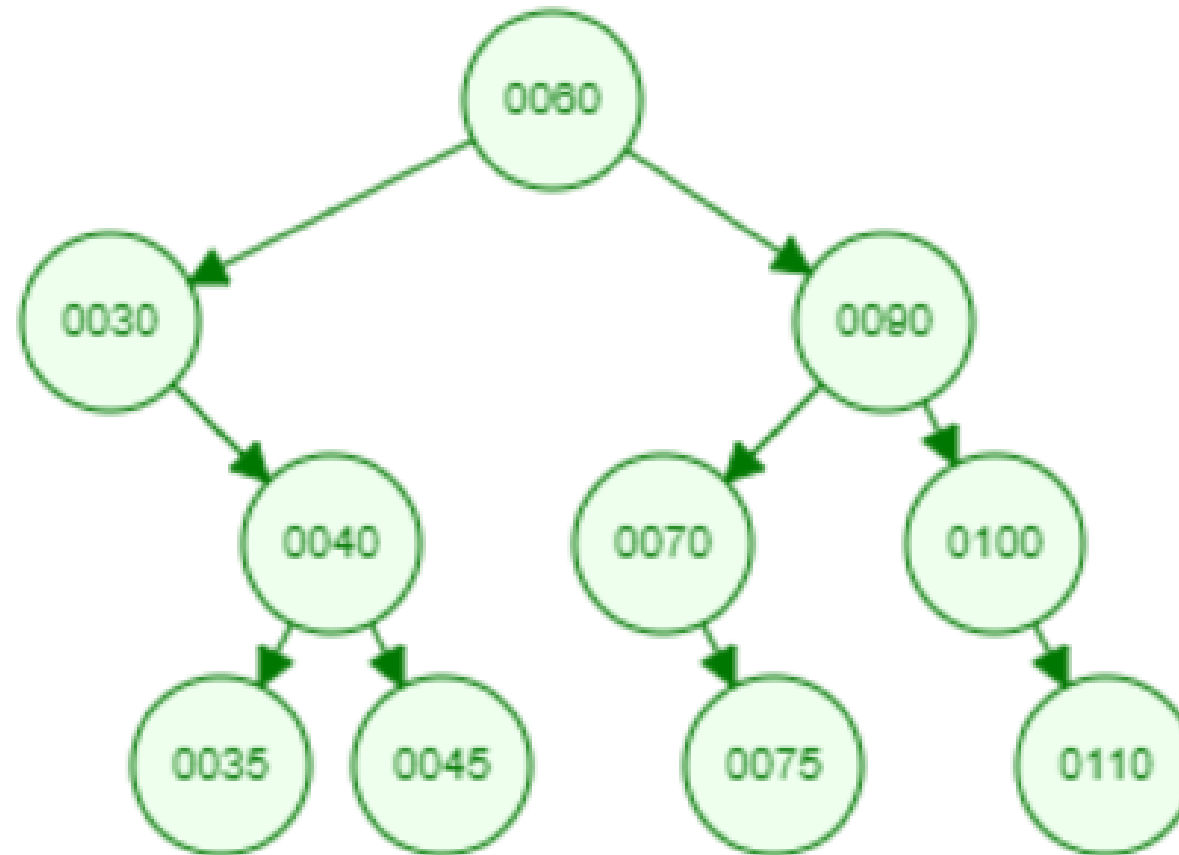
Rekurzivní funkce  $f(n)$  vytvoří vyvážený pravidelný binární strom s hloubkou  $\log_2(n)$ . Který z následujících výrazů nejlépe charakterizuje její asymptotickou složitost:

- a)  $\Omega(n)$
- b)  $\Theta(\log_2(n))$
- c)  $\Theta(n \cdot \log_2(n))$
- d)  $O(\log_2(n))$
- e)  $O(n)$

# Průchod BS do hloubky a do šířky

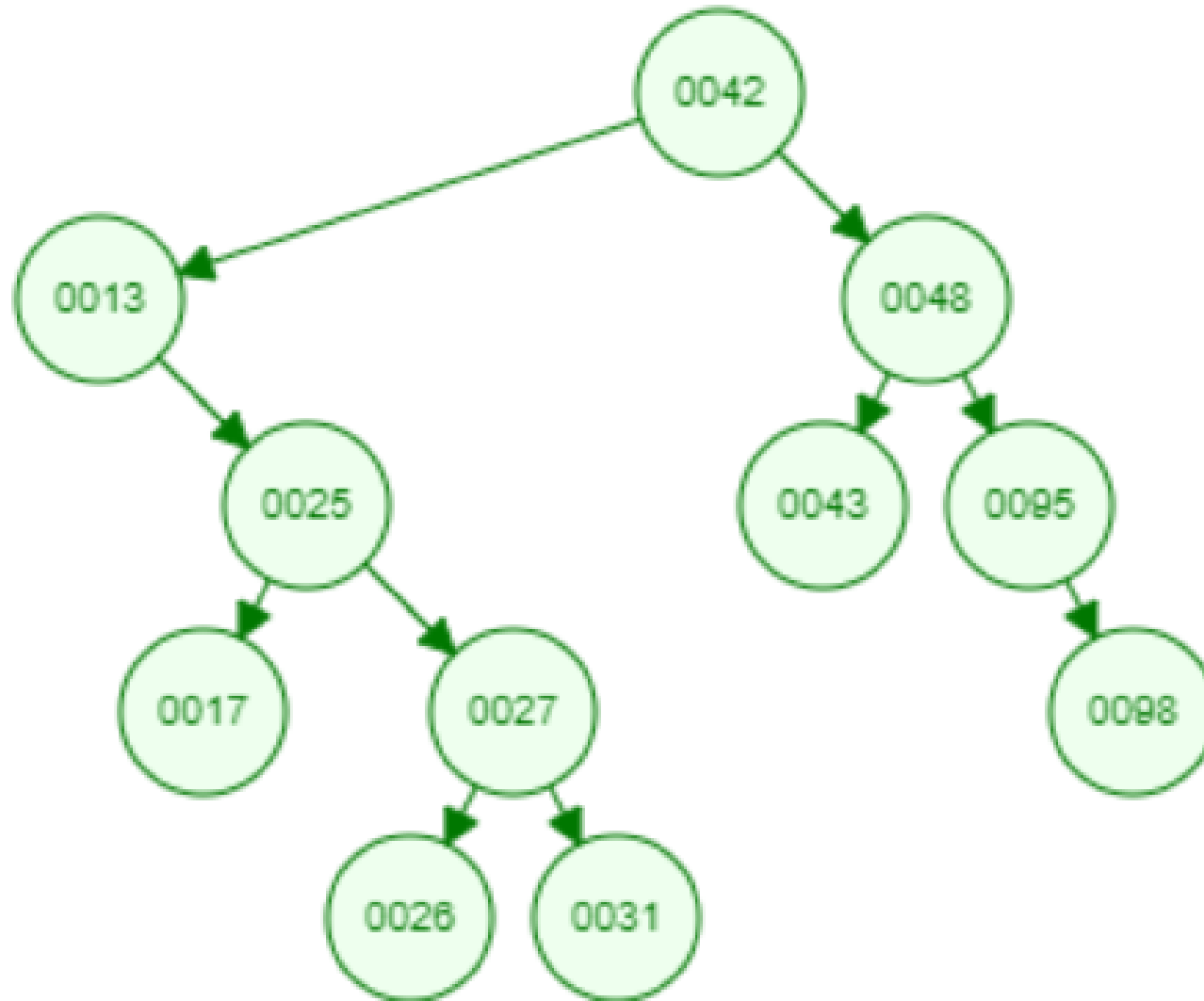
---

V jakém pořadí budou zpracovávány uzly následujícího stromu při jeho prohlédávání do hloubky a do šířky?



# Průchody stromem in-, pre- a postorder 1/2

---



## Průchody stromem in-, pre- a postorder 2/2

---

- a) K čemu jsou tyto typy průchodů vhodné?
- b) U jakého druhu stromů jsou totožné průchody in- a pre-order?
- c) U jakého druhu stromů jsou totožné průchody pre- a post-order? Jaká je hloubka tohoto stromu?
- d) Jaká je minimální hloubka binárního stromu s 300 a 600 uzly?
- e) Jaká je minimální hloubka ternárního stromu s 300 a 600 uzly?

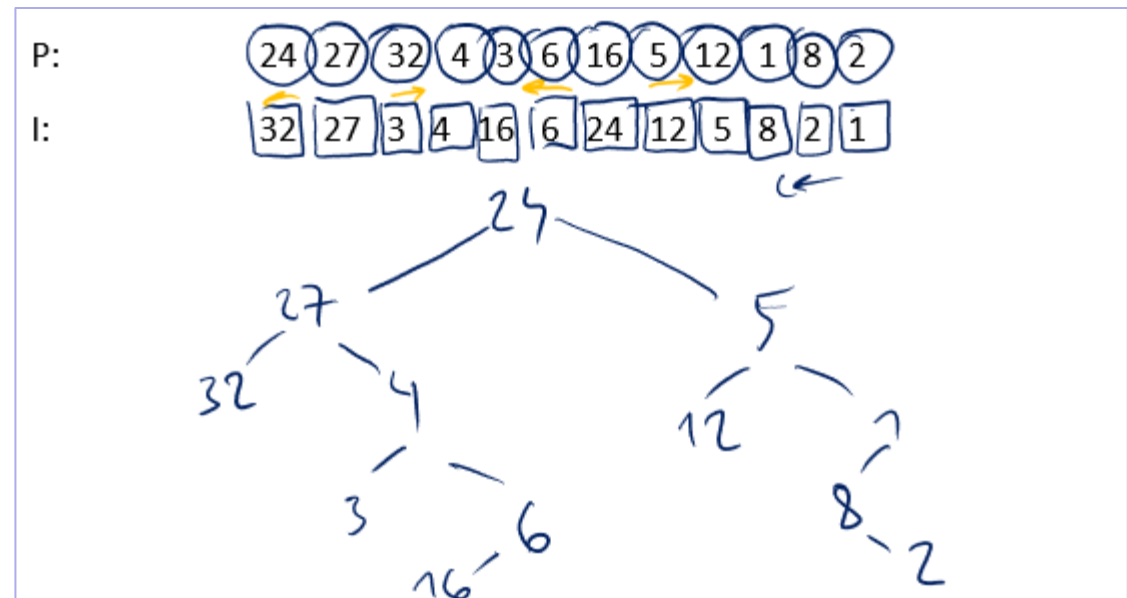
# Opakování: BS, konstrukce

Nakreslete binární strom, jehož preorder (P) a inorder (I) průchody generují následující sekvence:

P: 24 27 32 4 3 6 16 5 12 1 8 2

I: 32 27 3 4 16 6 24 12 5 8 2 1

Řešení



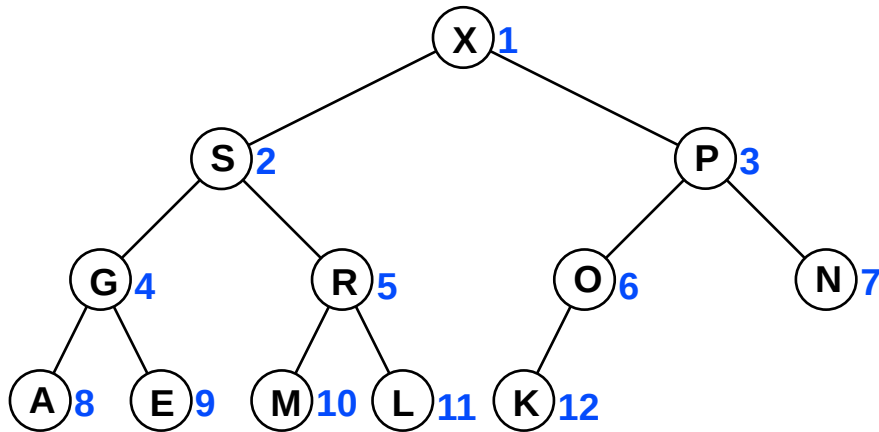
# Halda

- Halda
  - halda v poli
  - opravy haldy zdola nahoru a shora dolů
- Prioritní fronta realizovaná haldou
- Řazení pomocí prioritní fronty
- Tvorba haldy v poli
- Algoritmus řazení Heapsort

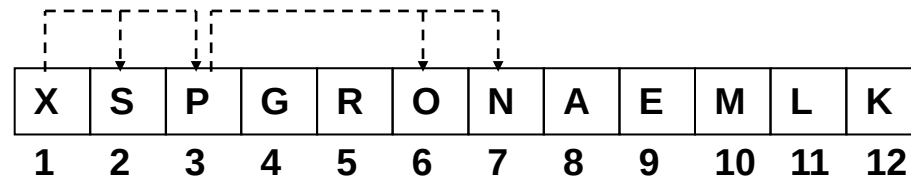


# Halda

- Halda: (zleva) úplný haldově uspořádaný binární strom

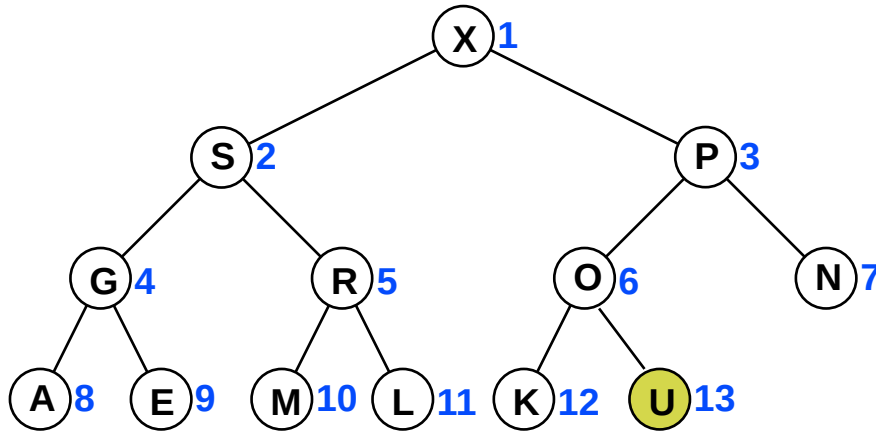


- Označme uzly indexy vzrůstajícími zleva doprava a shora dolů (počínaje 1)
- Platí: následníci uzlu s indexem  $k$  mají indexy  $2k$  (levý) a  $2k+1$  (pravý)
- Toho lze využít a uložit uzly haldy do pole:



## Vložení uzlu do haldy

- Nový uzel vložíme za uzel s nejvyšším indexem

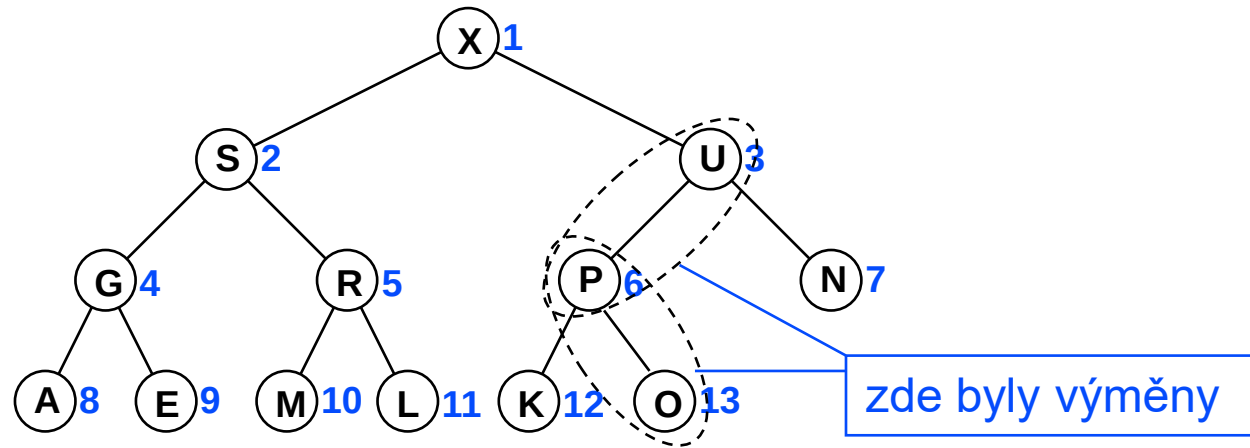


- Pak musíme haldu opravit zdola nahoru: pokud vložený uzel má větší klíč, než jeho rodič, vyměníme je a opakujeme pro vyšší úroveň ( $k = k/2$ ). Skončíme, když dojdeme ke kořenu nebo k uzlu, který nemá větší klíč, než jeho rodič.

```
void fixUp(T *a, int k) {  
    while (k>1 && a[k/2]<a[k]) {  
        swap(a[k], a[k/2]);  
        k = k/2;  
    }  
}
```

## Oprava haldy zdola nahoru

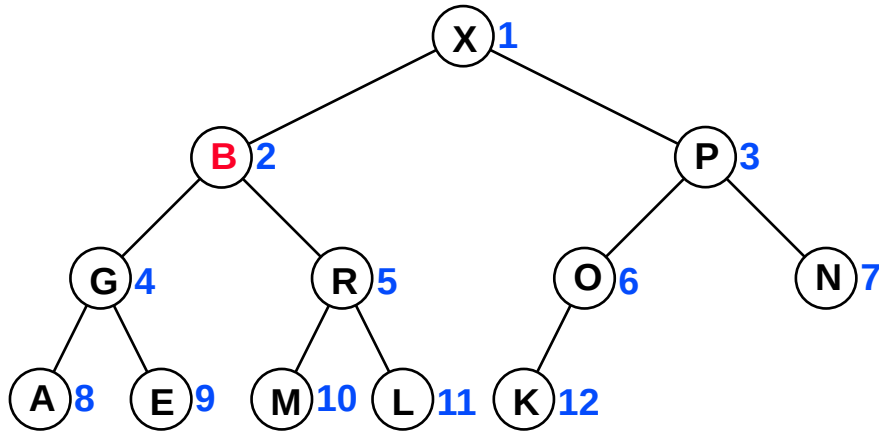
- Předchozí halda po opravě:



- Stejným způsobem opravíme haldu, když se **zvětší** klíč nějakého uzlu (v tom případě začneme s opravou u změněného uzlu)

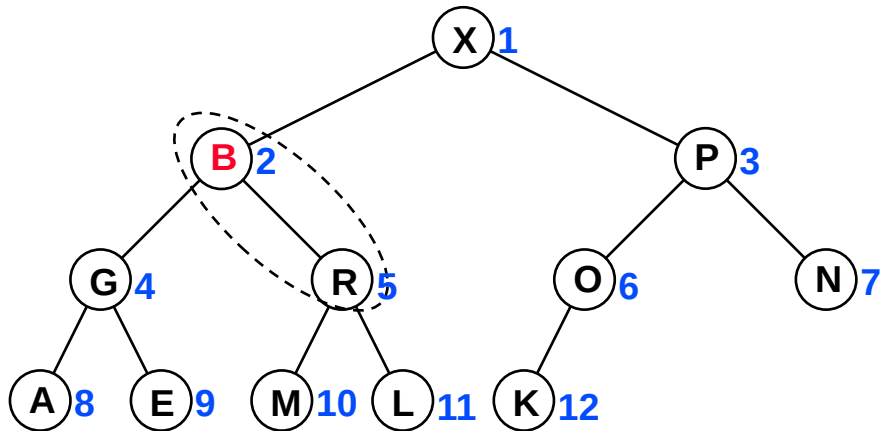
## Oprava haldy shora dolů

- Haldu opravíme algoritmem shora dolů, pokud se **zmenšil** klíč nějakého uzlu

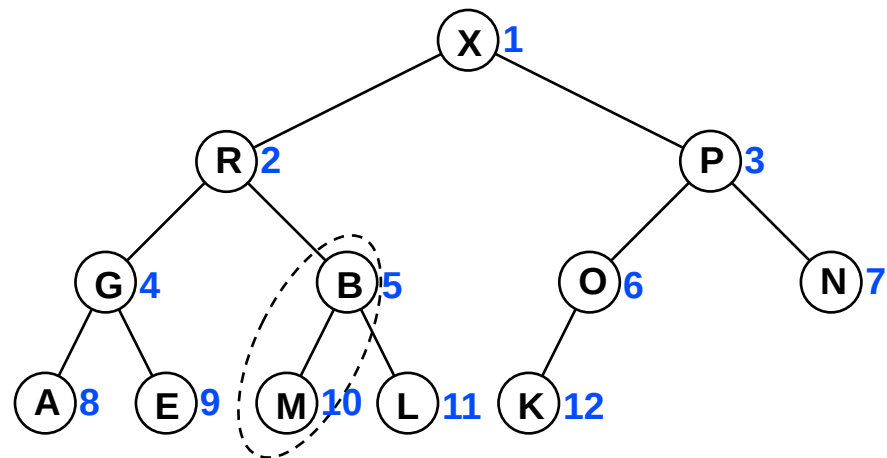


- Postup: Je-li to nutné, vyměníme změněný uzel s větším následníkem a opakujeme pro tohoto následníka. Skončíme, když změněný uzel je větší než oba následníci nebo když je to list.

## Oprava haldy shora dolů

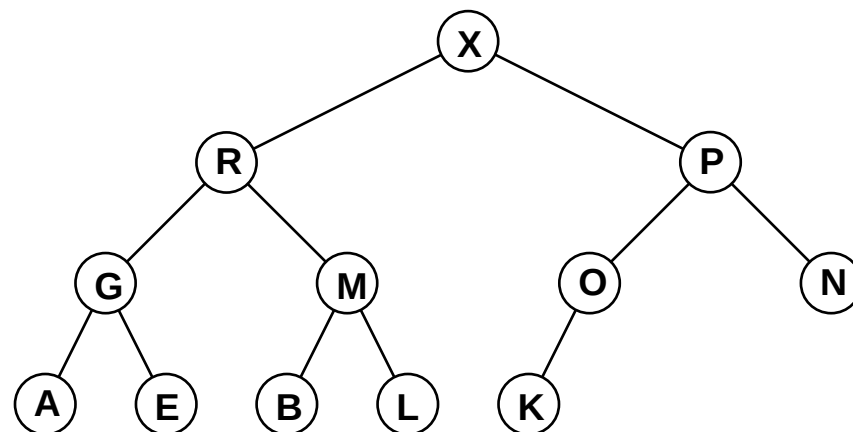


vyměníme u[2] a u[5]



vyměníme u[5] a u[10]

## Oprava haldy shora dolů

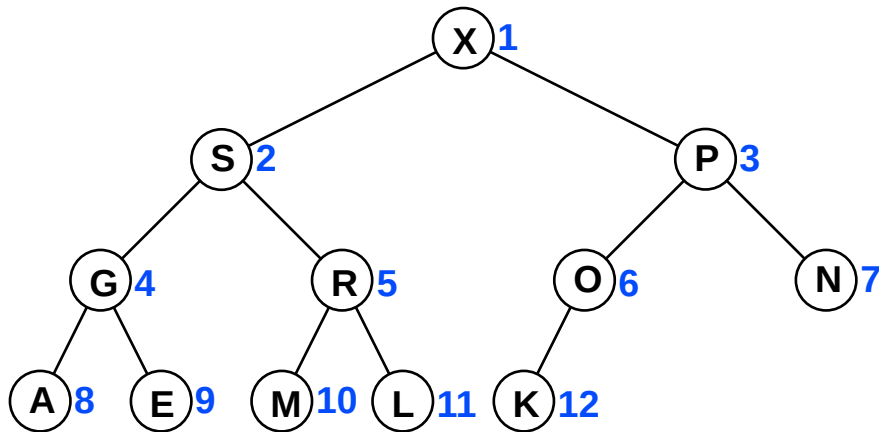


opravená halda

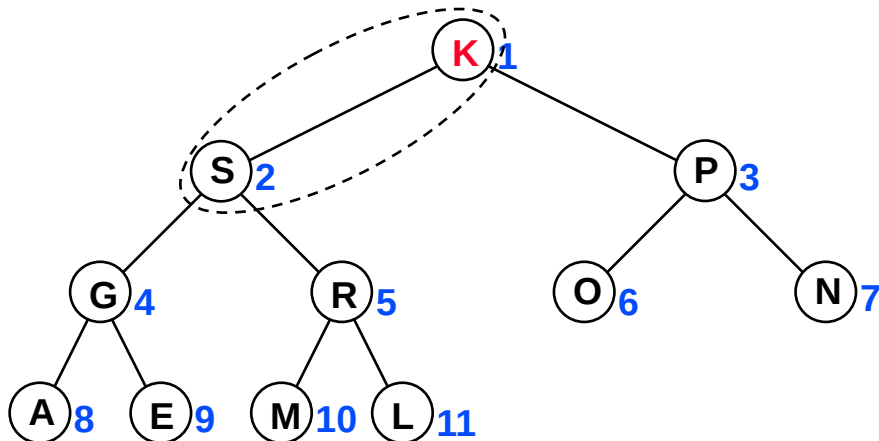
```
void fixDown(T *a, int k, int n) {  
    int j;  
    while (2*k <= n) {  
        j = 2 * k;  
        if (j < n && a[j] < a[j+1]) // větší ze synů  
            j++;  
        if (a[k] >= a[j])  
            break;  
        swap(a[k], a[j]); // záměna rodič - syn  
        k = j;  
    }  
}
```

## Odebrání kořene haldy

- Do kořene zkopírujeme poslední uzel, zmenšíme počet uzlů haldy a pak opravíme algoritmem shora dolů

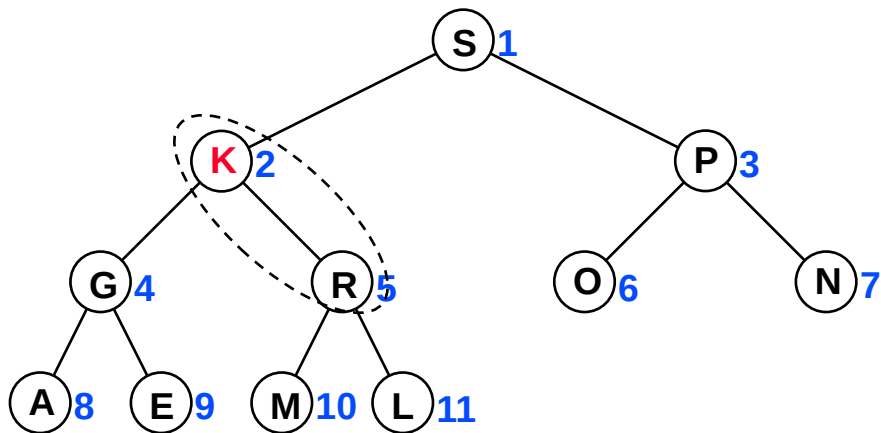


do  $u[1]$  zkopírujeme  $u[12]$   
a zmenšíme počet uzlů

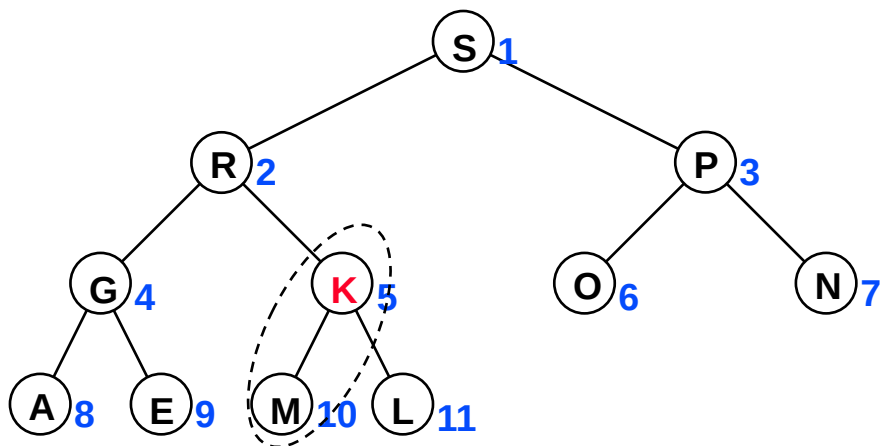


vyměníme  $u[1]$  a  $u[2]$

## Odebrání kořene haldy



vyměníme  $u[2]$  a  $u[5]$

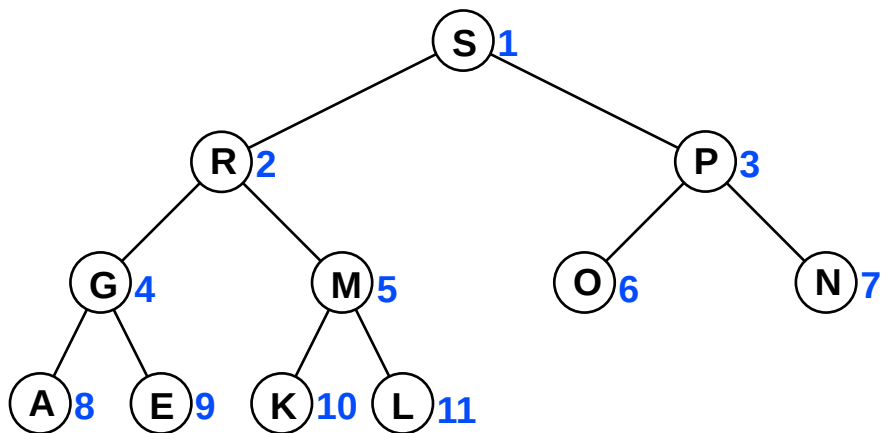


vyměníme  $u[5]$  a  $u[10]$



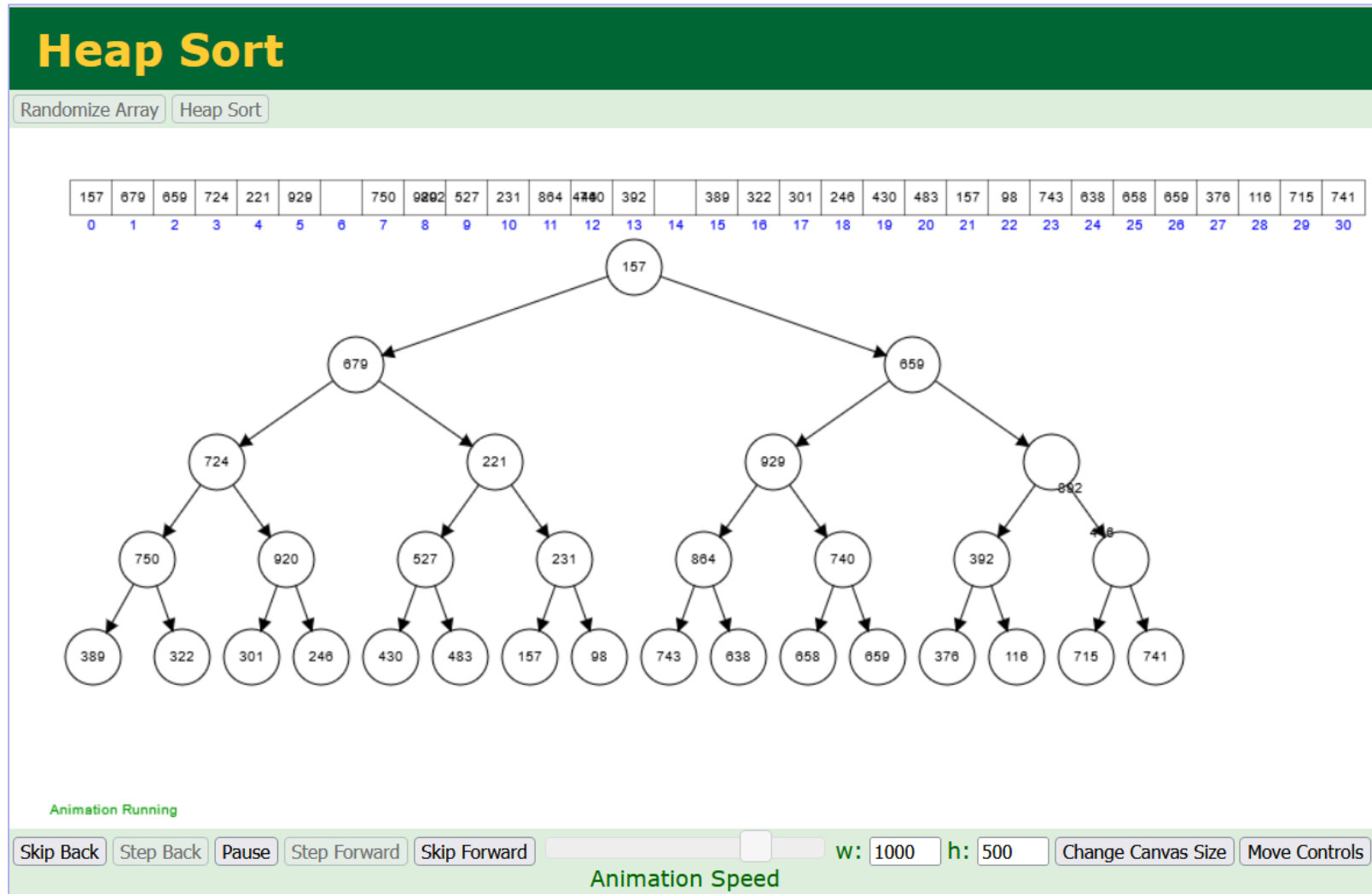
## Odebrání kořene haldy

- Opravená halda:



- !!! Algoritmy opravy haldy mají složitost  $O(\log_2 n)$  !!!

# Heap sort, animovaná ukázka



Zdroj: <https://www.cs.usfca.edu/~galles/visualization/HeapSort.html>

# Prioritní fronta pomocí haldy

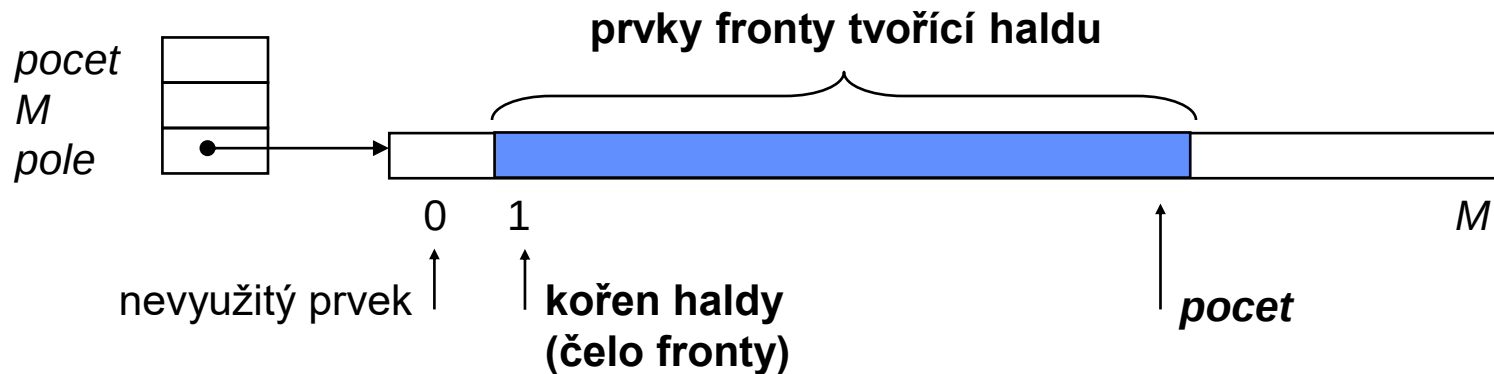
- **Prioritní fronta:**

- operace `remove()` vybírá prvek z čela fronty
- operace `add(x)` ukládá prvek `x` do fronty tak, aby v čele byl prvek s nejvyšší prioritou (největší prvek)

- Prvky fronty budeme ukládat do pole tak, aby tvořily haldu

- Realizace operací

- `add(x)` vložení nového uzlu do haldy
- `remove()` vrátí hodnotu kořenu haldy a kořen odebere



## Prioritní fronta pomocí šablon

```
template <class T>
void fixUp(T *a, int k) {
    while (k>1 && a[k/2]<a[k]) {
        swap(a[k], a[k/2]);
        k = k/2;
    }
}

template <class T>
void fixDown(T *a, int k, int n) {
    int j;
    while (2*k <= n) {
        j = 2 * k;
        if (j<n && a[j]<a[j+1])
            j++;
        if (a[k]>=a[j])
            break;
        swap(a[k], a[j]);
        k = j;
    }
}
```

# Šablona třídy PQueue

```
template <class T>
class PQueue {
public:
    PQueue(int = 40);
    ~PQueue();
    void add(const T&);
    T remove();
    bool empty() const;
private:
    int pocet;
    int M;
    T* pole;
    PQueue(const PQueue&);
    PQueue& operator=(const PQueue&);
};
```

# Šablony metod třídy PQueue

```
template <class T>           //konstruktor prázdné fronty
PQueue<T>::PQueue(int m) {
    pocet = 0;
    M = m;
    pole = new T[M+1];
}

template <class T>           //destruktor fronty
PQueue<T>::~~PQueue() {
    delete [] pole;
    pocet = 0;
    pole = 0;
}

template <class T>           //dotaz na prázdnou frontu
bool PQueue<T>::empty() const {
    return pocet==0;
}
```

## Šablony metod třídy PQueue

```
template <class T>          //vložení prvku x do fronty
void PQueue<T>::add(const T& x) {
    if (pocet==M) throw "plna fronta";
    // vlozeni hodnoty x do haldy 1..pocet
    pole[++pocet] = x;
    fixUp(pole, pocet);
}

template <class T>          //výběr prvku z fronty
T PQueue<T>::remove() {
    if (empty()) throw „odebrani z prazdne fronty";
    T res = pole[1];
    pole[1] = pole[pocet]; //poslední do kořene haldy
    pocet--;
    fixDown(pole, 1, pocet);
    return res;
}
```

## Použití šablony PQueue

```
#include "pqueue.h"
#include <iostream>
using namespace std;

int main() {
    int a[] = {1,10,3,7,3,12,5};
    int na = sizeof(a)/sizeof(int);
    try {
        Pqueue <int> q;
        // naplnění fronty
        for (int i=0; i<na; i++) q.add(a[i]);
        while (!q.empty()) // výběr a tisk prvků fronty
            cout << q.remove() << ' ';
        cout << endl;
    }
    catch (const char* s) {
        cout << "chyba: " << s << endl;
    }
    system("PAUSE");
    return 0;
}
```